

End-to-End Flight Data Engineering Pipeline using Azure & Databricks

Project-3

Student Name: Bhaskar Karri
Program: Azure Data Engineering

Table of Contents

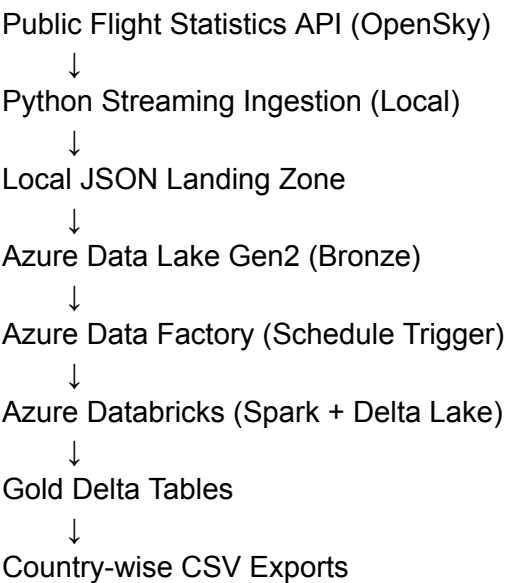
• Project Overview.....	2
• High-Level Architecture.....	2
• Technologies Used.....	2
• Step 1: Data Ingestion (Source → Local).....	3
• Step 2: Upload to Azure Data Lake (Bronze Layer).....	3
• Step 3: Event-Driven Orchestration.....	4
• Step 4: Data Transformation in Databricks.....	4
• Step 5: Gold Delta Table Creation.....	5
• Step 6: Latest Position per Aircraft (MERGE Logic).....	5
• Step 7: Data Quality & NULL Handling.....	6
• Step 8: Country-Wise CSV Export.....	6
• Step 9: Download & Delivery Automation.....	6
• Final Output.....	7
• Best Practices Applied.....	7
• Key Learnings & Outcomes.....	7
• Project_git_link.....	8

Project Overview

This project demonstrates the design and implementation of an end-to-end data engineering pipeline that ingests publicly available near real-time flight statistics from an external API, processes the data using Apache Spark and Delta Lake, and delivers curated, country-level CSV outputs for downstream consumption.

The solution follows a modern medallion architecture (Bronze → Gold) and emphasizes data quality, schema handling, ACID transactions, and automation readiness.

High-Level Architecture



Technologies Used

Layer	Technology
Source	OpenSky Network Public API
Ingestion	Python
Storage	Azure Data Lake Gen2
Orchestration	Azure Data Factory

Processing	Azure Databricks (PySpark)
Storage Format	Delta Lake
Output Format	CSV
Local Dev	VS Code

Step 1: Data Ingestion (Source → Local)

- Implemented a Python ingestion script to poll the OpenSky API at regular intervals.
- Each API response is stored as a timestamped JSON file, simulating near real-time micro-batch ingestion.
- Each file includes:
 - Batch metadata
 - Ingestion timestamp
 - Raw flight state arrays

Why this approach?

- Decouples ingestion from cloud dependencies
- Enables retry and local validation
- Mimics real streaming ingestion patterns

Step 2: Upload to Azure Data Lake (Bronze Layer)

- A Python uploader script moves JSON files from local storage to ADLS Gen2 (Bronze container).
- Files are stored as-is, preserving raw structure for traceability and replay.

- Local files are deleted only after successful upload.

Bronze Design Principle

Store immutable raw data exactly as received.

Step 3: Event-Driven Orchestration

- Azure Data Factory event trigger monitors the Bronze folder.
- On file arrival, it automatically triggers a Databricks notebook.
- This enables near real-time processing without polling.

Step 4: Data Transformation in Databricks

Raw Processing

- JSON files are read from ADLS Bronze.
- Nested flight states arrays are exploded into individual records.
- Positional arrays are mapped to named columns using the OpenSky API specification.

Schema Challenges Addressed

- Numeric fields (altitude, velocity, latitude, longitude) arrived as strings
- Some values were malformed or missing
- Applied safe casting (TRY_CAST) to prevent pipeline failures

Step 5: Gold Delta Table Creation

- Cleaned and transformed data is written to a Delta table (flight_positions) in the Gold layer.
- Table is partitioned and optimized for analytics use cases.
- Delta table is registered in the metastore for SQL access.

Delta Lake Features Used

- ACID transactions
- Schema enforcement
- MERGE (upsert) operations
- OPTIMIZE with Z-ORDER
- Time-travel readiness

Step 6: Latest Position per Aircraft (MERGE Logic)

Problem Encountered

- Multiple records per aircraft (icao24) caused Delta MERGE conflicts.

Solution

- Used window functions to deduplicate source data.
- Retained only the latest record per aircraft based on ingestion time.
- Ensured deterministic, conflict-free MERGE behavior.

Step 7: Data Quality & NULL Handling

- Profiled NULL values across all columns.
- Applied business rules:
 - Replaced missing callsigns with "UNKNOWN"
 - Set default values for missing altitude and velocity
 - Removed records with invalid latitude/longitude
- All updates performed using Delta UPDATE and DELETE operations.

Step 8: Country-Wise CSV Export

- Curated Gold data is exported as CSV files partitioned by origin_country.
- Output stored in ADLS under a dedicated export folder.
- Each country's data is isolated for easy downstream usage.

Step 9: Download & Delivery Automation

- Implemented a Python script to:
 - Download CSVs from ADLS
 - Extract country name from partition paths
 - Rename files to business-friendly names (e.g., flight_positions_India.csv)
 - Save outputs to a local delivery folder

This simulates data delivery to external consumers.

Final Output

data/outgoing_csvs/

```
|— flight_positions_India.csv  
|— flight_positions_United_States.csv  
|— flight_positions_Germany.csv
```

Each file contains clean, analytics-ready flight position data.

Best Practices Applied

- Clear separation of ingestion, processing, and delivery
- Idempotent and safe Delta MERGE operations
- Schema validation and type enforcement
- Data quality profiling before updates
- Event-driven orchestration
- Reproducible local development setup

Key Learnings & Outcomes

- Built a realistic streaming data pipeline using public APIs
- Gained hands-on experience with Delta Lake internals
- Learned to debug common production issues:
 - Schema inference problems
 - MERGE conflicts
 - Catalog vs path-based tables
- Delivered data in both analytics (Delta) and distribution (CSV) formats

Project_git_link= https://github.com/bhaskargit225/Flight_statistics_app